実験 I・測定基盤校正 PoC 設計および実行手順書

解析解を持つ線形ガウス系で、表現歪みの測定パイプラインを検証する

Toru Takahashi

v0.3 / 2026 年 8 月

この実験で確かめること

本 PoC の競合は、相関係数 ρ を決めた時点で解析的に定まる。したがって、理論に固有の仮説を検証するのではなく、ρ を走査した実測歪みが解析解を再現するかを確かめる。さらに標本数 n を走査し、残差の RMSE が $n^{-1/2}$、MSE が n^{-1} のオーダーで縮小するかを測る。これにより、有限標本の推定誤差と、標本を増やしても残る実装バイアスを分ける。

推奨環境は Google Colab の標準 CPU である。GPU、有料クラウド、オープンモデル、大容量データは使わない。同梱コードは NumPy のみに依存し、検証済みの短時間設定は通常の CPU で数秒〜数分を想定する。

目次

1 研究全体のどこを試すのか 2

2 最小環境の設計 2

2.1 同じ観測に含まれる、二つの保存要求 2

2.2 小さな共有表現モデル 3

2.3 解析解 4

3 実験条件と事前の判定基準 4

3.1 短時間設定 4

3.2 測定基盤の校正条件 4

4 検証済み実行の結果例 5

5 Google Colab で実行する（推奨） 6

5.1 必要なもの 6

5.2 クリックする順序 7

6 自分の PC で実行する 7
6.1 macOS の手順 7
6.2 Windows 11 の手順 9
7 出力を読む 9
8 トラブル対応 10
9 個人研究者の実施順序と費用 10
10 実験ログと次の意思決定 11
11 後続研究へ何を引き継ぐか 11
公式環境資料 12

1 研究全体のどこを試すのか

構成的検証計画は、(I) 情報取得・保存の競合、(II) 履歴依存、(III) 文脈モードではないレジーム性、(IV) 異なる履歴からの構造的反復、の順に進む。本 PoC が扱うのはその最初の一部である。

$$\boxed{\text{本 PoC} = \text{解析解を用いた、表現歪みの測定基盤の校正}} \tag{1}$$

観測・注意予算 B_A、初期学習履歴、準安定レジーム、人物指標の保存などはまだ導入しない。理由は、最初から複数の機構を入れると、結果がどの操作から生じたのか判定できなくなるからである。

この PoC が証明しないこと

人間の世界モデルが実際にこの学習機構をもつこと、心理志向／客観志向が実在すること、履歴依存的な志向性が生じること、複数の準安定レジームが発達することは、この段階では未検証である。また、一般に情報保存トレードオフが実在することへの経験的支持でもない。

2 最小環境の設計

2.1 同じ観測に含まれる、二つの保存要求

各データ点に、保存すべき情報 $A \in \mathbb{R}^3$ と $B \in \mathbb{R}^3$ を用意する。A と B は無関係ではなく、一部の予測可能な構造を共有しつつ、それぞれに固有の情報を持つ。

$$A \sim \mathcal{N}(0, I), \tag{2}$$

$$B = \rho A + \sqrt{1-\rho^2}\,\varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I), \qquad 0 \le \rho < 1. \tag{3}$$

主実験では $\rho = 0.35$ とし、校正実験で複数の ρ を走査する。

例えるなら、同じ出来事に含まれる「人物と関係の変化」と「人物を入れ替えても保存される構造」のように、共通因子はあるが同じ情報ではない二つの対象を抽象化したものである。ただし現段階で A や B を心理志向、客観志向と呼ばない。これらは内容を持たない合成変数である。

モデルが受け取る観測 $X \in \mathbb{R}^6$ は、A と B を連結し、ランダムな直交変換 Q で混合したものである。

$$X = [A, B]Q + \epsilon_{\text{obs}}. \tag{4}$$

これにより、モデルは入力の前半を A、後半を B としてそのまま取り出せない。観測から共有状態表現を学習する必要がある。

2.2 小さな共有表現モデル

モデルは、1 つの共有エンコーダ E と 2 つの読み出しヘッド H_A, H_B からなる。

$$Z = E(X), \qquad Z \in \mathbb{R}^{C_R}, \tag{5}$$
$$\widehat{A} = H_A(Z), \tag{6}$$
$$\widehat{B} = H_B(Z). \tag{7}$$

すべて線形層で実装する。これはニューラルネットとしては最も小さい部類である。非線形層、Attention、Transformer、強化学習を入れないのは、競合が複雑なアーキテクチャの偶然ではなく、共有表現の有限性から生じるかを最初に切り分けるためである。

学習目的は、二つの正規化平均二乗誤差を重み付きで最小化することである。

$$D_A = \frac{\mathbb{E}\|A - \widehat{A}\|^2}{\mathrm{Var}(A)}, \qquad D_B = \frac{\mathbb{E}\|B - \widehat{B}\|^2}{\mathrm{Var}(B)}, \tag{8}$$
$$\mathcal{L}_\lambda = \lambda D_A + (1 - \lambda) D_B. \tag{9}$$

$D = 0$ はほぼ完全な再構成、$D = 1$ は平均値を返すだけの基準水準である。$\lambda = 1$ は A のみ、$\lambda = 0$ は B のみを学習する。$\lambda = 0.5$ は二つを同じ重みで保存しようとする条件である。

ただし、歪みは学習中のヘッドをそのまま使って測らない。理論上の定義

$$D_k = \inf_{\hat{\xi}_k} \mathbb{E}[d_k(\xi_k, \hat{\xi}_k(Z))] \tag{10}$$

に近づけるため、エンコーダの学習後に Z を固定し、A, B の両方に切片付き最小二乗読み出しを当て直す。これにより、$\lambda = 0$ または 1 で更新されなかったヘッドの性能を、表現から失われた情報と誤認しない。

2.3　解析解

$C_R = 3$ で $\lambda = 0.5$ のとき、問題は 3 つの同型な 2 次元問題に分解できる。観測ノイズの標準偏差を σ とすると、各要求の母集団歪みは

$$D_A^*(\rho) = D_B^*(\rho) = 1 - \frac{(1+\rho)^2}{2(1+\rho+\sigma^2)} \simeq \frac{1-\rho}{2} \tag{11}$$

となる。後半は σ が小さいときの近似である。$\rho = 0.35, \sigma = 0.02$ なら理論値は約 0.3252 である。したがって、旧版の閾値は競合を発見する検定ではなく、実装が解析的予測を再現するかの工学的な検査であった。

3　実験条件と事前の判定基準

3.1　短時間設定

表 1　初回に実行する `--quick` 条件

項目	設定
学習データ	2,048 例
評価データ	1,024 例（学習に使わない）
共有表現次元 C_R	3（制約）、6（拡張対照）
ρ	0.35（主実験）、0, 0.2, 0.35, 0.5, 0.7, 0.9（校正走査）
観測ノイズ σ	0.02
λ	0, 0.25, 0.5, 0.75, 1
乱数 seed	0, 1, 2
学習回数	500
必要ハードウェア	CPU、メモリ数 GB 程度

$C_R = 3$ は A または B を単独で保存するには足りるが、固有情報を含む両者の全部を同時保存するには狭い。$C_R = 6$ は容量を拡張した対照である。

3.2　測定基盤の校正条件

閾値は人間の実在に関する統計検定ではなく、次の実験へ進むかを決める工学的なゲートである。

1. **解析曲線の再現：** ρ 走査における実測平均歪みと解析解の RMSE が 0.03 未満、最大絶対誤差が 0.05 未満である。

2. **最適読み出し：**$\lambda = 0, 1$ の未学習ヘッドではなく、固定した Z から当て直した最小二乗読み出しで D_A, D_B が集計される。
3. **容量対照：**$C_R = 6$ で両方の歪みが小さく、主実験の計算とデータ生成に明らかな不具合がない。
4. **標本数スケーリング：**複数 seed による RMSE が標本数とともに縮小し、log–log 傾きが -0.5 に大きく反しない。MSE の対応する理論的傾きは -1 である。

校正が失敗した場合、本設計では「情報的トレードオフが存在しなかった」とは解釈しない。データ生成、前処理、勾配学習の収束、最適読み出しの実装を先に点検する。

4　検証済み実行の結果例

同梱コードの `--quick` 設定を CPU で実行した。結果は次のとおりである。

表 2　$\lambda = 0.5, \rho = 0.35$ における歪み（3 seed 平均）

C_R	D_A	D_B	実測平均	解析値
3	0.3277	0.3161	0.3219	0.3252
6	0.0004	0.0004	0.0004	—

$C_R = 3$ で A を優先した条件の D_A は 0.0004、B を優先した条件の D_B もほぼ 0.0004 であった。最適読み出しを当て直した結果、反対側の歪みは 1.0 ではなく約 0.875 となり、理論値 $1 - \rho^2 = 0.8775$ と整合した。旧版の 1.0 は、読み出しヘッドが更新されていないことによる人工物であった。

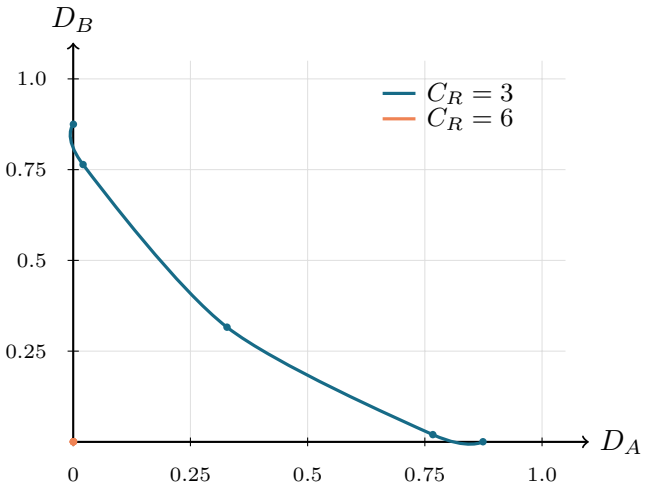


図 1　最適読み出しで測った二つの保存歪み。左下ほど両者を良く保存する。

六点の RMSE は 0.0086、最大絶対誤差は 0.0109 であり、校正基準を満たした。これが示すのは、競合の新しい発見ではない。勾配学習が閉形式の予測へ収束し、後続実験で使う歪み測定が数値的に妥当であることである。

$\rho = 0$ では、$\lambda = 0.5$ の目的関数を同値にする表現配分が複数存在する。そのため、平均歪みは安定

表 3　ρ 走査による解析解との校正（3 seed 平均）

ρ	実測平均歪み	解析値	絶対誤差
0.00	0.4956	0.5002	0.0046
0.20	0.4108	0.4002	0.0106
0.35	0.3361	0.3252	0.0109
0.50	0.2607	0.2502	0.0105
0.70	0.1583	0.1502	0.0081
0.90	0.0534	0.0502	0.0032

しても、個別の D_A, D_B は初期値に応じて大きく変わりうる。これは誤差ではなく、対称な解集合が一意に選択されない縮退である。

表 4　標本数校正（quick、8 seed）

学習標本数 n	実測平均歪み	符号付きバイアス	RMSE
1,024	0.3290	0.0038	0.0103
2,048	0.3301	0.0049	0.0086
4,096	0.3221	−0.0031	0.0041
8,192	0.3238	−0.0014	0.0038

RMSE の log–log 傾きは −0.537、MSE の傾きは −1.073 であった。ここで注意すべきなのは、誤差の縮小を「$1/n$」と一括りにしないことである。標準的には、RMSE は $n^{-1/2}$、その二乗である MSE は n^{-1} のオーダーをもつ。この結果は、解析解との差が固定的な実装バイアスよりも、有限標本の推定誤差として振る舞っていることを支持する。

5　Google Colab で実行する（推奨）

Google Colab は、ブラウザ上で Python ノートブックを実行でき、基本的な環境構築を必要としない。無料枠の計算資源は固定的に保証されないが、本 PoC は CPU で十分なため、GPU の割当は要求しない [1, 2]。

5.1　必要なもの

- Google アカウントと Chrome 等のブラウザ
- `colab_runner.ipynb`
- `minimal_poc.py`

5.2 クリックする順序

1. ブラウザで https://colab.research.google.com/ を開く。
2.「ファイル」または表示されるノートブック選択画面から「ノートブックをアップロード」を選び、colab_runner.ipynb を開く。
3. メニューの「ランタイム」→「ランタイムのタイプを変更」を開き、ハードウェア アクセラレータが「なし」であることを確認する。
4. 最初のコードセル左側の三角ボタンを押す。ファイル選択が表示されたら、minimal_poc.py を選ぶ。
5. 二番目のコードセルを実行する。「進捗 30/30」の後に、相関 ρ と標本数 n の校正走査が続く。校正判定が出るまでタブを閉じない。
6. 三番目のセルで自動判定と図を確認する。
7. 四番目のセルを実行し、experiment_i_minimal_poc_results.zip をダウンロードする。Colab の一時環境は切断で消えるため、ZIP の保存を忘れない。

初回は --quick のまま実行する。
--full は条件数と seed 数を増やす。操作の成功と出力の読み方を確認するまで、実行する必要はない。

---

6 自分の PC で実行する

Google Colab で一度成功した後に、必要な場合だけローカル環境を作る。Python 公式ドキュメントと Python Packaging User Guide は、プロジェクトごとの仮想環境 venv を使う手順を示している [3, 4]。これにより、PC 全体の Python 環境を汚さずに実験用パッケージを分離できる。

6.1 macOS の手順

1. https://www.python.org/downloads/ から Python 3 を導入する。既に使用できる場合は不要である。
2.「ターミナル」を開き、Finder から PoC フォルダをターミナルへドラッグして、保存場所を確認する。
3. ターミナルで次を一行ずつ実行する。

```
cd "PoCフォルダの実際の場所"
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

```
python minimal_poc.py --quick --output-dir poc_results
```

次回以降は、PoC フォルダで `source .venv/bin/activate` と最後の実行コマンドを使う。

6.2 Windows 11 の手順

1. Python 公式サイトから Python 3 を導入する。インストーラに「Add Python to PATH」が表示された場合は有効にする。
2. PoC フォルダを開き、エクスプローラのアドレス欄に `powershell` と入力して Enter を押す。
3. PowerShell で次を一行ずつ実行する。

```
py -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
python minimal_poc.py --quick --output-dir poc_results
```

PowerShell が仮想環境の有効化を拒否する場合は、コマンドプロンプトを開き、次を実行する。

```
.venv\Scripts\activate.bat
```

7 出力を読む

実行後、指定した `poc_results` フォルダに次が作られる。

ファイル	用途
`result_report.md`	自動判定と主要数値。まずこれを読む。
`tradeoff.svg`	横軸 D_A、縦軸 D_B の図。ブラウザで開ける。
`rho_calibration.svg`	ρ に対する実測歪みと解析曲線の重ね合わせ。
`rho_calibration.csv`	ρ ごとの実測平均、標準偏差、解析値、絶対誤差。
`rho_calibration_raw.csv`	ρ ごとの seed 別 D_A, D_B。$\rho = 0$ の縮退もここから調べられる。
`sample_size_calibration.svg`	標本数と RMSE の log–log プロット。
`sample_size_calibration.csv`	標本数ごとのバイアス、RMSE、MSE。
`sample_size_calibration_raw.csv`	標本数・seed ごとの生の歪みと誤差。
`summary_results.csv`	seed 平均と標準偏差。Excel、Google Sheets、テキストエディタで開ける。
`raw_results.csv`	各 seed・各条件の生の結果。後の統計解析に使う。

ファイル	用途
`metadata.json`	データ数、学習条件、Python と NumPy の版、実行時間。

図は左下ほど良い。$C_R = 3$ の曲線が λ に応じて右下から左上へ移るなら、一方を精密に保存すると他方が失われる。$C_R = 6$ の中間点が左下に移るなら、その競合はデータ本来の学習不可能性ではなく、共有表現容量の制約に依存している。

8 トラブル対応

現象	対応
`No module named numpy`	Colab では新しいセルで `!pip install numpy` を一度実行する。ローカルでは仮想環境を有効化後、`python -m pip install -r requirements.txt` を実行する。
`minimal_poc.py` がない	Colab の最初のセルを再実行し、正しい名前のファイルを選ぶ。
実行が途中で止まった	Colab の「ランタイム」→「セッションを再起動」後、最初から実行する。まず `--quick` を使う。
自動判定が「校正未達」	`metadata.json`、`rho_calibration.csv`、画面のエラーを保存する。本設計では環境ではなく、実装、前処理、収束、読み出しを先に点検する。
ローカルのインストールで迷った	ローカル環境構築を一度中断し、Colab 版で実験手順と結果を先に確認する。

9 個人研究者の実施順序と費用

1. **0 円・1 時間目：** Colab で `--quick` を実行し、ZIP を保存する。コードは変更しない。
2. **0 円・2 時間目：** CSV と図を開き、判定基準と数値の対応を理解する。
3. **0 円・半日：** `--full` を実行し、フォルダ名に日付を付けて保存する。
4. **必要になった後：** ローカル PC に Python 環境を作る。GPU はまだ購入しない。
5. **共同研究への移行：** 合成データの意味ある環境化、観測予算 B_A の導入、履歴実験へ進む段階で、研究室と役割分担する。

最初の PoC のために PC や GPU を購入する必要はない。計算量が増えるのは、B_A, C_R, λ の格子、

seed、環境を大幅に増やし、数百〜数千本の学習を管理する段階である。そこまでは、お金よりも実験の判定基準を固めることが重要である。

10　実験ログと次の意思決定

各実行で、次の最小ログを残す。

記録項目	記入内容
実行日時	
実行場所	Colab / macOS / Windows
実行モード	quick / full
使用ファイル版	v0.3
自動判定	校正成功 / 校正未達 / エラー
主要数値	解析解との RMSE・最大絶対誤差
変更した点	なし／変更内容
保存先	ZIP または結果フォルダ

次へ進む条件

`--quick` と `--full` で解析曲線が再現されたら、このコードを歪み測定の回帰テストとして固定する。次は、観測予算 B_A と表現容量 C_R を独立に操作する実験、または人物置換に対する不変性／等変性を持つ環境へ進む。

11　後続研究へ何を引き継ぐか

本 PoC は、予め与えた λ と ρ の下で、解析的に定まる歪みを実装が再現するかを確かめる。したがって、λ そのものが学習履歴から発達することも、非自明なトレードオフが発見されたことも示さない。

次の研究では、学習初期だけ A または B に関する経験を増やし、後半は完全に同じ環境へ戻す。その後も観測、表現、評価、後続学習の配分差が残るなら、予め与えた重みではなく、学習された志向の候補を扱える。

さらに観測予算 B_A を導入するときは、「見たが圧縮で失われた情報」と「そもそも観測されなかった情報」を分ける。また、パレート前線の凸性・非凸性を調べる場合、重み付き和の λ 走査だけでは非凸部分を検出できない。片方の歪みを ϵ 以下に制約して他方を最小化する ϵ 制約法は、別実験として実装する。

公式環境資料

参考文献

[1] Google Research, “Google Colab FAQ,” https://research.google.com/colaboratory/intl/en-GB/faq.html.

[2] Google Research, “Colab resource limits and runtime availability,” https://research.google.com/colaboratory/faq.html?authuser=8.

[3] Python Software Foundation, “venv — Creation of virtual environments,” https://docs.python.org/3/library/venv.html.

[4] Python Packaging Authority, “Installing packages using pip and virtual environments,” https://packaging.python.org/en/latest/guides/installing-using-pip-and-virtual-environments/.